The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...
Module 2: WHY THIS WO...
2.1 Video Text Guide (Series ...
2.2 One Line of Python Trigge...
2.3 How a 1953 Word Game E...
2.4 The Ladder That Explains ...
2.5 Clawdbot (Moltbot) Has 1...
2.6 Video as Code: My AI Ani...
2.7 From Nazi Psychology to ...
Module 3: FOLDER ARCHI...

2

3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...

4.2: Claude Code in Practice

What You'll Get From This

You will understand what makes Claude Code different from the chat interface, see the iteration loop in action, and walk away with a real deliverable built from your own files.

Video: Session 2 — Claude Code in Practice

> Coming Soon

The Lesson

The real power

Claude Code is not smarter than Claude Desktop. It is the same model. The difference is what it can reach. Claude Code can see your work, touch your files, and iterate on its own output. That changes everything about how you work with it.

The loop looks like this: Read → Think → Write → Check → Adjust. Claude reads your files, thinks about the task, writes output, checks its own work, and adjusts. In the chat interface, that loop runs once and stops. In Claude Code, it runs until the

Example: meeting notes

Here is a task that shows the difference clearly.

You have 15 meeting notes. You want every decision made this quarter, organized by project.

Module 4: CLAUDE CODE ...

4.1: Install and First Use

4.2: Claude Code in Practice

4.3: Claude Desktop as a Thin...

4.4 Making Claude Understan...

4.5: Where This Goes

SECTION 5: WHERE TO G...

The Desktop approach: Open file 1. Copy the text. Paste it into Claude. Ask for analysis. Repeat 15 times. Then manually combine all 15 responses into one document. About an hour of work.

The Claude Code approach: One prompt: "Read every file in /meeting-notes/ and summarize all decisions, organized by project." One prompt. 15 files. 90 seconds.

The model gives you the same quality analysis either way. The difference is how much manual work you do around it. Claude Code eliminates the copy-paste layer entirely.

Token math

Claude's context window is roughly 200K tokens. A typical meeting note is about 2,000 tokens. 15 notes is about 30,000 tokens. Well within the window.

In Desktop, you paste one file at a time and lose context between chats. Each conversation starts fresh. You are working against the architecture.

In Claude Code, it loads what it needs from your file system and keeps full context across the task. You are working with the architecture.

This is the same principle from **2.3 (How a 1953 Word Game Explains AI Memory)**: the context window is working memory. Claude Code lets you fill that working memory from your actual files instead of from your clipboard.

Tips that save hours

Be specific. "Analyze this" is weak. "Identify the three largest cost items and format as a table" is strong. The more specific

Tell it where to put the output. "Save the result as summary.md in this folder." Claude Code can write files. Let it. If your output lives in a chat window, you still have to copy it somewhere. If Claude writes it directly to a file, it is already where it needs to be.

Output wrong? Do not start over. Say what is wrong. "The summary missed the budget decisions from the March 14 meeting. Add those." Claude iterates. That is the whole point. Starting over throws away all the context Claude has built up about your task.

Claude Code is best for tasks that involve reading and writing files. That is where it shines. If your task is "help me think through a decision," Desktop might be better (that is Session 3). If your task is "read these 15 files and produce a deliverable," Claude Code wins every time.

Your first real task

Pick one of these (or something similar from your own work):

- "I have documents or notes. Summarize them."
- "I have a spreadsheet or CSV. What is interesting in here?"
- "I have a report or proposal. Find the gaps."
- "I have a folder of files. Organize them by [criteria]."

Open your folder in VS Code. Click Claude in the sidebar. Describe your task specifically. Watch it work. Iterate if something is off.

You should walk away with a real deliverable from your own files. Something you can actually use tomorrow.

Go Deeper

- The prompting framework from **1.3 How to Structure Any Prompt** applies directly here. Identity and context come from your files. Task, constraints, and output format go in

2.3: How a 1953 Word Game Explains AI Memory.

- If your task involves processing lots of files across different types of work, you may want the folder architecture from **Section 3** to organize things before you start.

Next Up: 4.3 Claude Desktop as a Thinking Partner

Claude Code is for building. But sometimes you need to think before you build. Session 3 shows you how to use Claude Desktop for planning, pressure-testing ideas, and finding blind spots, and then take that clarity into Code to execute.

Matthew Creamer
24d • Lesson Check-Ins

Foundations 4.2 Check-In

You just ran Claude Code on your own files. Vote below, then drop what you made in the comments. What was the

Poll 50 members have voted

15 17 New comment 5h ago